Ввод [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats as st
import warnings
warnings.filterwarnings('ignore')
```

Ввод [2]:

```
calls = pd.read_csv('/datasets/calls.csv')
internet = pd.read_csv('/datasets/internet.csv')
messages = pd.read_csv('/datasets/messages.csv')
tariffs = pd.read_csv('/datasets/tariffs.csv')
users = pd.read_csv('/datasets/users.csv')
```

1. Основные тезисы

Клиент - компания «Мегалайн»

Основных тарифных плана два: «Смарт» и «Ультра»

Задача - сделать предварительный анализ тарифов на выборке из 500 клиентов, основываясь на их поведении как абонентов

Цель - выяснить, какой тариф приносит больше денег, для корректировки рекламного бюджета

2. Знакомство с данными

2.1. Звонки

Смотрим данные по звонкам

Ввод [3]:

```
calls.head()
```

Out[3]:

	id	call_date	duration	user_id
0	1000_0	2018-07-25	0.00	1000
1	1000_1	2018-08-17	0.00	1000
2	1000_2	2018-06-11	2.85	1000
3	1000_3	2018-09-21	13.80	1000
4	1000_4	2018-12-15	5.18	1000

Изучаем информацию о них

Ввод [4]:

```
calls.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 202607 entries, 0 to 202606
Data columns (total 4 columns):
 #   Column     Non-Null Count   Dtype  
---  ------     --------------   -----  
 0   id         202607 non-null  object 
 1   call_date  202607 non-null  object 
 2   duration   202607 non-null  float64
 3   user_id    202607 non-null  int64  
dtypes: float64(1), int64(1), object(2)
memory usage: 6.2+ MB
```

Явных пропусков нет

Посмотрим describe

Ввод [5]:

```
calls.describe()
```

Out[5]:

	duration	user_id
count	202607.000000	202607.000000
mean	6.755887	1253.940619
std	5.843365	144.722751
min	0.000000	1000.000000
25%	1.300000	1126.000000
50%	6.000000	1260.000000
75%	10.700000	1379.000000
max	38.000000	1499.000000

Минимум 0, максимум 38 минут, выглядит разумно

Ввод [6]:

```
print(calls.call_date.min())
print(calls.call_date.max())
```

```
2018-01-01
2018-12-31
```

Даты логичные - с 1 января по 31 декабря, задействован весь год

Сделаем культурно - переведем столбец с датами в соответствующий формат

Ввод [7]:

```
calls['call_date'] = pd.to_datetime(calls.call_date)
```

И округлим звонки до минут хитрой схемой: целые числа оставим как есть, нецелые - вверх

Ввод [8]:

```
calls['rounded_duration'] = np.ceil(calls.duration).astype(int)
```

Ввод [9]:

```
calls.head()
```

Out[9]:

	id	call_date	duration	user_id	rounded_duration
0	1000_0	2018-07-25	0.00	1000	0
1	1000_1	2018-08-17	0.00	1000	0
2	1000_2	2018-06-11	2.85	1000	3
3	1000_3	2018-09-21	13.80	1000	14
4	1000_4	2018-12-15	5.18	1000	6

Выглядит разумно

2.2. Интернет

Проводим те же манипуляции

Ввод [10]:

```
internet.head()
```

Out[10]:

	Unnamed: 0	id	mb_used	session_date	user_id
0	0	1000_0	112.95	2018-11-25	1000
1	1	1000_1	1052.81	2018-09-07	1000
2	2	1000_2	1197.26	2018-06-25	1000
3	3	1000_3	550.27	2018-08-22	1000
4	4	1000_4	302.56	2018-09-24	1000

Замечаем непонятный столбец, и методом логических выводов понимаем, что это какой-то счетчик пользователей, смысла в данном случае не имеющий. Удаляем

Ввод [11]:

```
internet = internet.drop('Unnamed: 0', axis=1)
```

Ввод [12]:

```
internet.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 149396 entries, 0 to 149395
Data columns (total 4 columns):
 #   Column        Non-Null Count   Dtype  
---  ------        --------------   -----  
 0   id            149396 non-null  object 
 1   mb_used       149396 non-null  float64
 2   session_date  149396 non-null  object 
 3   user_id       149396 non-null  int64  
dtypes: float64(1), int64(1), object(2)
memory usage: 4.6+ MB
```

Вновь ни одной ошибки

Ввод [13]:

```
internet.describe()
```

Out[13]:

	mb_used	user_id
count	149396.000000	149396.000000
mean	370.192426	1252.099842
std	278.300951	144.050823
min	0.000000	1000.000000
25%	138.187500	1130.000000
50%	348.015000	1251.000000
75%	559.552500	1380.000000
max	1724.830000	1499.000000

Минимум 0, максимум 1.7 Гб за сессию, вроде ничего сверхъестественного

Ввод [14]:

```
print(internet.session_date.min())
print(internet.session_date.max())
```

```
2018-01-01
2018-12-31
```

Даты так же ведут себя прилично, следовательно, причесываем их в нужный формат

Ввод [15]:

```
internet['session_date'] = pd.to_datetime(internet.session_date)
```

2.3. Сообщения

И вновь по знакомой схеме

Ввод [16]:

```
messages.head()
```

Out[16]:

	id	message_date	user_id
0	1000_0	2018-06-27	1000
1	1000_1	2018-10-08	1000
2	1000_2	2018-08-04	1000
3	1000_3	2018-06-16	1000
4	1000_4	2018-12-05	1000

Ввод [17]:

```
messages.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 123036 entries, 0 to 123035
Data columns (total 3 columns):
 #   Column        Non-Null Count   Dtype 
---  ------        --------------   ----- 
 0   id            123036 non-null  object
 1   message_date  123036 non-null  object
 2   user_id       123036 non-null  int64 
dtypes: int64(1), object(2)
memory usage: 2.8+ MB
```

Снова ни одной ошибки, грузчикам премию, со времен первого проекта они сильно прокачались

Ввод [18]:

```
print(messages.message_date.min())
print(messages.message_date.max())
```

```
2018-01-02
2018-12-31
```

Вновь усредняем тип данных

Ввод [19]:

```
messages['message_date'] = pd.to_datetime(messages.message_date)
```

2.4. Тарифы

Ввод [20]:

```
tariffs.head()
```

Out[20]:

	messages_included	mb_per_month_included	minutes_included	rub_monthly_fee	rub_per_gb
0	50	15360	500	550	200
1	1000	30720	3000	1950	150

Ничего лишнего

2.5. Пользователи

Ввод [21]:

```
users.head()
```

Out[21]:

	user_id	age	churn_date	city	first_name	last_name	reg_date	tariff
0	1000	52	NaN	Краснодар	Рафаил	Верещагин	2018-05-25	ultra
1	1001	41	NaN	Москва	Иван	Ежов	2018-11-01	smart
2	1002	59	NaN	Стерлитамак	Евгений	Абрамович	2018-06-17	smart
3	1003	23	NaN	Москва	Белла	Белякова	2018-08-17	ultra
4	1004	68	NaN	Новокузнецк	Татьяна	Авдеенко	2018-05-14	ultra

Ввод [22]:

```
users.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   user_id     500 non-null    int64 
 1   age         500 non-null    int64 
 2   churn_date  38 non-null     object
 3   city        500 non-null    object
 4   first_name  500 non-null    object
 5   last_name   500 non-null    object
 6   reg_date    500 non-null    object
 7   tariff      500 non-null    object
dtypes: int64(2), object(6)
memory usage: 31.4+ KB
```

Пропущенные данные с датами отключения – хороший знак, значит, никто не ушел

Ввод [23]:

```
print(users.age.min())
print(users.age.max())
```

```
18
75
```

Возраст соответствует действительности

Ввод [24]:

```
users.city.sort_values().unique()
```

Out[24]:

```
array(['Архангельск', 'Астрахань', 'Балашиха', 'Барнаул', 'Белгород',
       'Брянск', 'Владивосток', 'Владикавказ', 'Владимир', 'Волгогра
д',
       'Волжский', 'Вологда', 'Воронеж', 'Грозный', 'Екатеринбург',
       'Иваново', 'Ижевск', 'Иркутск', 'Казань', 'Калининград', 'Калуг
а',
       'Кемерово', 'Киров', 'Кострома', 'Краснодар', 'Красноярск',
       'Курган', 'Курск', 'Липецк', 'Магнитогорск', 'Махачкала', 'Моск
ва',
       'Мурманск', 'Набережные Челны', 'Нижневартовск', 'Нижний Новгор
од',
       'Нижний Тагил', 'Новокузнецк', 'Новороссийск', 'Новосибирск',
       'Омск', 'Оренбург', 'Орёл', 'Пенза', 'Пермь', 'Петрозаводск',
       'Подольск', 'Ростов-на-Дону', 'Рязань', 'Самара',
       'Санкт-Петербург', 'Саранск', 'Саратов', 'Севастополь', 'Смолен
ск',
       'Сочи', 'Ставрополь', 'Стерлитамак', 'Сургут', 'Тамбов', 'Твер
ь',
       'Тольятти', 'Томск', 'Тула', 'Тюмень', 'Улан-Удэ', 'Ульяновск',
       'Уфа', 'Хабаровск', 'Химки', 'Чебоксары', 'Челябинск', 'Черепов
ец',
       'Чита', 'Якутск', 'Ярославль'], dtype=object)
```

В городах нет дубликатов

Ввод [25]:

```
users.tariff.unique()
```

Out[25]:

```
array(['ultra', 'smart'], dtype=object)
```

Тарифы в порядке. Необычайно чистые данные

Ввод [26]:

```
users['churn_date'] = pd.to_datetime(users.churn_date)
users['reg_date'] = pd.to_datetime(users.reg_date)
```

3. Количество звонков и минут по месяцам

Для начала, выделим месяцы в звонках

Ввод [27]:

```
calls['month'] = calls.call_date.dt.month
```

Ввод [28]:

```
calls.head()
```

Out[28]:

	id	call_date	duration	user_id	rounded_duration	month
0	1000_0	2018-07-25	0.00	1000	0	7
1	1000_1	2018-08-17	0.00	1000	0	8
2	1000_2	2018-06-11	2.85	1000	3	6
3	1000_3	2018-09-21	13.80	1000	14	9
4	1000_4	2018-12-15	5.18	1000	6	12

Отлично, теперь можно строить сводную по месяцам

Ввод [29]:

```
calls_pivot = calls.pivot_table(index=['month', 'user_id'],
                                values=['rounded_duration'],
                                aggfunc=['sum', 'count'])
```

Ввод [30]:

```
calls_pivot.head()
```

Out[30]:

		sum	count
		rounded_duration	rounded_duration
month	**user_id**		
	1005	95	15
	1032	380	61
1	**1045**	9	1
	1055	551	64
	1071	194	24

4. Количество сообщений по месяцам

Так же выделяем месяцы в отдельный столбец

Ввод [31]:

```
messages['month'] = messages.message_date.dt.month
```

И делаем сводную таблицу

Ввод [32]:

```
messages_pivot = messages.pivot_table(index=['month', 'user_id'],
                                      values=['message_date'],
                                      aggfunc=['count'])
```

Ввод [33]:

```
messages_pivot.head()
```

Out[33]:

		count
		message_date
month	user_id	
	1005	7
	1032	38
1	1045	5
	1055	30
	1071	16

5. Объем интернет-трафика по месяцам

Та же участь для месяцев

Ввод [34]:

```
internet['month'] = internet.session_date.dt.month
```

Та же сводная таблица

Ввод [35]:

```
internet_pivot = internet.pivot_table(index=['month', 'user_id'],
                                      values=['mb_used'],
                                      aggfunc=['sum'])
```

И еще округлим суммарные данные за месяц до круглых Гб с применением нашей хитрой методики

Ввод [36]:

```
internet_pivot['rounded_gb_used'] = (internet_pivot['sum']['mb_used']
                                     .apply(
                                         lambda x: int(x) if x%1024 == 0 else int(x)
                                     )
                                    )
```

Ввод [37]:

```
internet_pivot.head()
```

Out[37]:

		sum mb_used	rounded_gb_used
month	user_id		
	1005	2727.97	3
	1032	24127.69	24
1	**1045**	2346.71	3
	1055	17090.88	17
	1071	15604.34	16

6. Помесячная выручка

```
С каждого пользователя

Нам перед глазами пригодятся две таблички
```

Ввод [38]:

```
tariffs
```

Out[38]:

	messages_included	mb_per_month_included	minutes_included	rub_monthly_fee	rub_per_gb
0	50	15360	500	550	200
1	1000	30720	3000	1950	150

Ввод [39]:

```
users.head()
```

Out[39]:

	user_id	age	churn_date	city	first_name	last_name	reg_date	tariff
0	1000	52	NaT	Краснодар	Рафаил	Верещагин	2018-05-25	ultra
1	1001	41	NaT	Москва	Иван	Ежов	2018-11-01	smart
2	1002	59	NaT	Стерлитамак	Евгений	Абрамович	2018-06-17	smart
3	1003	23	NaT	Москва	Белла	Белякова	2018-08-17	ultra
4	1004	68	NaT	Новокузнецк	Татьяна	Авдеенко	2018-05-14	ultra

6.1. Звонки

Выведем так же сводную по звонкам

Ввод [40]:

```
calls_pivot.head()
```

Out[40]:

		sum	count
		rounded_duration	rounded_duration
month	user_id		
	1005	95	15
	1032	380	61
1	1045	9	1
	1055	551	64
	1071	194	24

Добавим в нее необходимую информацию

Тариф:

Ввод [41]:

```
calls_pivot['tariff_name'] = calls_pivot.merge(users, on='user_id', how='left')['tar
```

Включенное количество минут:

Ввод [42]:

```
calls_pivot['minutes_included'] = (calls_pivot
                                   .merge(tariffs, on='tariff_name', how='left')['mi
                                   .tolist())
```

Ежемесячную плату:

Ввод [43]:

```
calls_pivot['rub_monthly_fee'] = calls_pivot.merge(tariffs, on='tariff_name', how='l
```

И плату за превышение лимита:

Ввод [44]:

```
calls_pivot['rub_per_minute'] = calls_pivot.merge(tariffs, on='tariff_name', how='le
```

Рассчитаем тех, кто таки превысил и поплатился

Ввод [45]:

```
calls_pivot['fee_over'] = calls_pivot['minutes_included'] - calls_pivot['sum']['rour
calls_pivot['fee_over'] = calls_pivot['fee_over'] * calls_pivot['rub_per_minute']
calls_pivot['fee_over'] = calls_pivot['fee_over'].apply(lambda x: abs(x) if x < 0 el
```

Ввод [46]:

```
calls_pivot
```

Out[46]:

		sum	count	tariff_name	minutes_included	rub_monthly_
		rounded_duration	rounded_duration			
month	**user_id**					
	1005	95	15	smart	500	
	1032	380	61	smart	500	
1	**1045**	9	1	smart	500	
	1055	551	64	smart	500	
	1071	194	24	smart	500	
...	**...**	...	...	...	...	
	1494	418	62	smart	500	
	1495	1031	155	ultra	3000	1
12	**1496**	518	80	smart	500	
	1497	545	71	smart	500	
	1499	492	69	smart	500	

3174 rows × 7 columns

6.2. Сообщения

Ввод [47]:

```
messages_pivot.head()
```

Out[47]:

month	user_id	count message_date
1	1005	7
	1032	38
	1045	5
	1055	30
	1071	16

Здесь логика такая же - и такой же алгоритм

Ввод [48]:

```
messages_pivot['tariff_name'] = messages_pivot.merge(users, on='user_id')['tariff'].
```

Ввод [49]:

```
messages_pivot['messages_included'] = messages_pivot.merge(tariffs, on='tariff_name'
```

Ввод [50]:

```
messages_pivot['rub_monthly_fee'] = messages_pivot.merge(tariffs, on='tariff_name')[
```

Ввод [51]:

```
messages_pivot['rub_per_message'] = messages_pivot.merge(tariffs, on='tariff_name')[
```

Ввод [52]:

```
messages_pivot['fee_over'] = messages_pivot['messages_included'] - messages_pivot['c
messages_pivot['fee_over'] = messages_pivot['fee_over'] * messages_pivot['rub_per_me
messages_pivot['fee_over'] = messages_pivot['fee_over'].apply(lambda x: abs(x) if x
```

Ввод [53]:

```
messages_pivot
```

Out[53]:

		count	tariff_name	messages_included	rub_monthly_fee	rub_per_messa
		message_date				
month	user_id					
	1005	7	smart	50	550	
	1032	38	smart	50	550	
1	1045	5	smart	50	550	
	1055	30	smart	50	550	
	1071	16	smart	50	550	
...	...	...	...	...	...	
	1493	78	smart	1000	1950	
	1495	84	smart	1000	1950	
12	1496	51	smart	1000	1950	
	1497	17	smart	1000	1950	
	1499	66	ultra	1000	1950	

2717 rows × 6 columns

6.3. Интернет

Ввод [54]:

```
internet_pivot.head()
```

Out[54]:

		sum	rounded_gb_used
		mb_used	
month	user_id		
	1005	2727.97	3
	1032	24127.69	24
1	1045	2346.71	3
	1055	17090.88	17
	1071	15604.34	16

Ввод [55]:

```
internet_pivot['tariff_name'] = internet_pivot.merge(users, on='user_id')['tariff'].
```

Ввод [56]:

```
internet_pivot['gb_per_month_included'] = (internet_pivot
                                           .merge(tariffs, on='tariff_name')['mb_per_mo
```

Ввод [57]:

```
internet_pivot['rub_monthly_fee'] = internet_pivot.merge(tariffs, on='tariff_name')[
```

Ввод [58]:

```
internet_pivot['rub_per_gb'] = internet_pivot.merge(tariffs, on='tariff_name')['rub_
```

Ввод [59]:

```
internet_pivot['fee_over'] = internet_pivot['gb_per_month_included'] - internet_pivo
internet_pivot['fee_over'] = internet_pivot['fee_over'] * internet_pivot['rub_per_gb
internet_pivot['fee_over'] = internet_pivot['fee_over'].apply(lambda x: abs(x) if x
```

Ввод [60]:

```
internet_pivot.head()
```

Out[60]:

		sum mb_used	rounded_gb_used	tariff_name	gb_per_month_included	rub_monthly_fe
month	user_id					
	1005	2727.97	3	smart	15	55
	1032	24127.69	24	smart	15	55
1	**1045**	2346.71	3	smart	15	55
	1055	17090.88	17	smart	15	55
	1071	15604.34	16	smart	15	55

6.4. Общая выручка

Выведем основные таблицы и соединим уже по абонентам

Ввод [61]:

```
calls_pivot_by_user = calls_pivot.pivot_table(index=['user_id', 'month'],
                                              values=['fee_over'],
                                              aggfunc='sum')
calls_pivot_by_user.head(10)
```

Out[61]:

		fee_over
user_id	**month**	
1000	**5**	0
	6	0
	7	0
	8	0
	9	0
	10	0
	11	0
	12	0
1001	**11**	0
	12	0

Ввод [62]:

```
messages_pivot_by_user = messages_pivot.pivot_table(index=['user_id', 'month'],
                                                    values=['fee_over'],
                                                    aggfunc='sum')
messages_pivot_by_user.head(10)
```

Out[62]:

		fee_over
user_id	month	
1000	5	0
	6	30
	7	75
	8	93
	9	21
	10	69
	11	0
	12	0
1002	6	0
	7	0

Ввод [63]:

```
internet_pivot_by_user = internet_pivot.pivot_table(index=['user_id', 'month'],
                                                    values=['fee_over'],
                                                    aggfunc='sum')
internet_pivot_by_user.head(10)
```

Out[63]:

		fee_over
user_id	month	
1000	5	0
	6	1600
	7	0
	8	0
	9	0
	10	0
	11	0
	12	0
1001	11	0
	12	0

Присоеденим их всех в одну таблицу

Ввод [64]:

```
all_users_pivot = (calls_pivot_by_user
                   .join(messages_pivot_by_user,
                         lsuffix='_calls',
                         rsuffix='_messages',
                         how='outer')
                   .join(internet_pivot_by_user,
                         rsuffix='_internet',
                         how='outer'))

all_users_pivot['fee_over_sum'] = (all_users_pivot['fee_over_calls'] +
                                   all_users_pivot['fee_over_messages'] +
                                   all_users_pivot['fee_over'])

all_users_pivot.columns = ['fee_over_calls',
                           'fee_over_calls',
                           'fee_over',
                           'fee_over_sum']

all_users_pivot.head()
```

Out[64]:

		fee_over_calls	fee_over_calls	fee_over	fee_over_sum
user_id	**month**				
	5	0.0	0.0	0.0	0.0
	6	0.0	30.0	1600.0	1630.0
1000	**7**	0.0	75.0	0.0	75.0
	8	0.0	93.0	0.0	93.0
	9	0.0	21.0	0.0	21.0

Ввод [65]:

```
all_users_pivot.index.get_level_values(0).nunique()
```

Out[65]:

```
498
```

Кажется, у нас два человека не пользовались никакими услугами. Будем лояльны и тоже добавим их в эту таблицу

Ввод [66]:

```
users.query('user_id not in @all_users_pivot.index.get_level_values(0)')
```

Out[66]:

	user_id	age	churn_date	city	first_name	last_name	reg_date	tariff
128	1128	51	2018-12-23	Волжский	Ксения	Агаева	2018-12-15	ultra
371	1371	50	2018-12-30	Омск	Ириней	Трофимов	2018-12-25	smart

Ввод [67]:

```
temp_users = pd.DataFrame({'user_id': (users
                                       .query('user_id not in @all_users_pivot.index
                                       .tolist()),
                           'month': [1, 1],
                           'fee_over_calls':[0, 0],
                           'fee_over_messages': [0, 0],
                           'fee_over': [0, 0],
                           'fee_over_sum': [0, 0]})
temp_users = temp_users.pivot_table(index=['user_id', 'month'], values=['fee_over_ca
                                                                        'fee_over_calls',
                                                                        'fee_over',
                                                                        'fee_over_sum'],
                                    aggfunc='sum')
temp_users.columns = all_users_pivot.columns
temp_users
```

Out[67]:

		fee_over_calls	fee_over_calls	fee_over	fee_over_sum
user_id	**month**				
1128	**1**	0	0	0	0
1371	**1**	0	0	0	0

Ввод [68]:

```
all_users_pivot = pd.concat([all_users_pivot, temp_users], axis=0)
```

Так как мы соединяли всех принудительно, появились пропущенные значения, которые означают отсутствие оплаты. Так и запишем

Ввод [69]:

```
all_users_pivot = all_users_pivot.fillna(0)
```

7. Анализ полученных данных

7.1. Минуты разговора

Посмотрим основные известные данные

Для smart

Ввод [70]:

```
print('Среднее значение:')
print(calls_pivot[calls_pivot.tariff_name == 'smart']['sum']['rounded_duration'].mea
print('Дисперсия:')
print(calls_pivot[calls_pivot.tariff_name == 'smart']['sum']['rounded_duration'].var
print('Стандартное отклонение:')
print(calls_pivot[calls_pivot.tariff_name == 'smart']['sum']['rounded_duration'].var
print('Доступно бесплатно:', '\n500')
```

```
Среднее значение:
419.0629779577148

Дисперсия:
35844.1895513438

Стандартное отклонение:
189.3256177894154

Доступно бесплатно:
500
```

Для ultra

Ввод [71]:

```
print('Среднее значение:')
print(calls_pivot[calls_pivot.tariff_name == 'ultra']['sum']['rounded_duration'].mea
print('Дисперсия:')
print(calls_pivot[calls_pivot.tariff_name == 'ultra']['sum']['rounded_duration'].var
print('Стандартное отклонение:')
print(calls_pivot[calls_pivot.tariff_name == 'ultra']['sum']['rounded_duration'].var
print('Доступно бесплатно:', '\n3000')
```

```
Среднее значение:
545.4511041009464

Дисперсия:
94203.41839614809

Стандартное отклонение:
306.9257538821858

Доступно бесплатно:
3000
```

Построим гистограмму

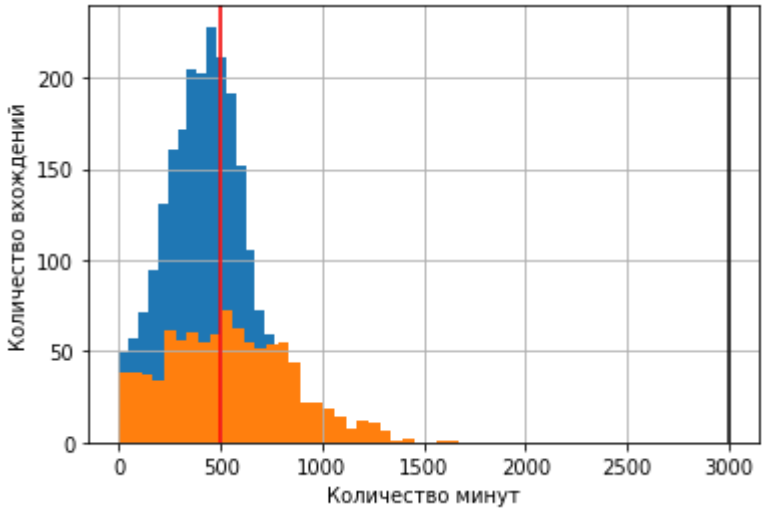
Ввод [72]:

```
calls_pivot[calls_pivot.tariff_name == 'smart']['sum']['rounded_duration'].hist(bins
calls_pivot[calls_pivot.tariff_name == 'ultra']['sum']['rounded_duration'].hist(bins

plt.axvline(x=int(tariffs.query('tariff_name == "smart"')['minutes_included']), colo
plt.axvline(x=int(tariffs.query('tariff_name == "ultra"')['minutes_included']), colo

plt.xlabel('Количество минут')
plt.ylabel('Количество вхождений')

plt.show()
```



АНАЛИТИКА ПО SMART

Заметна небольшая отрицательная скошенность

Пользователям доступно 500 минут, и пик как раз приходится недалеко от этого значения

Однако, можно заметить, что большая часть людей превышает свой лимит

АНАЛИТИКА ПО ULTRA

Здесь есть очень легкая отрицательная скошенность

Однако, видно, что большинство людей не превышают в использовании ~870 минут разговора

До 3000 минут, которые включены в тариф, не доходит вообще никто

7.2. Сообщения

Для smart

Ввод [73]:

```
print('Среднее значение:')
print(messages_pivot[messages_pivot.tariff_name == 'smart']['count']['message_date']
print('Дисперсия:')
print(messages_pivot[messages_pivot.tariff_name == 'smart']['count']['message_date']
print('Стандартное отклонение:')
print(messages_pivot[messages_pivot.tariff_name == 'smart']['count']['message_date']
print('Доступно бесплатно:', '\n50')
```

```
Среднее значение:
45.04058272632674

Дисперсия:
1215.8609237622834

Стандартное отклонение:
34.86919734898243

Доступно бесплатно:
50
```

Для ultra

Ввод [74]:

```
print('Среднее значение:')
print(messages_pivot[messages_pivot.tariff_name == 'ultra']['count']['message_date']
print('Дисперсия:')
print(messages_pivot[messages_pivot.tariff_name == 'ultra']['count']['message_date']
print('Стандартное отклонение:')
print(messages_pivot[messages_pivot.tariff_name == 'ultra']['count']['message_date']
print('Доступно бесплатно:', '\n1000')
```

```
Среднее значение:
45.87169811320755

Дисперсия:
1260.4721828810418

Стандартное отклонение:
35.503129198438856

Доступно бесплатно:
1000
```

Гистограмма

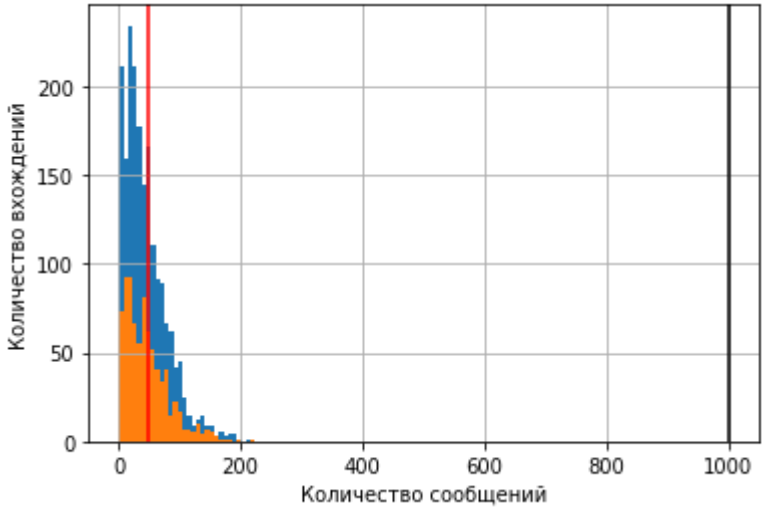
Ввод [75]:

```
messages_pivot[messages_pivot.tariff_name == 'smart']['count']['message_date'].hist(
messages_pivot[messages_pivot.tariff_name == 'ultra']['count']['message_date'].hist(

plt.axvline(x=int(tariffs.query('tariff_name == "smart"')['messages_included']), col
plt.axvline(x=int(tariffs.query('tariff_name == "ultra"')['messages_included']), col

plt.xlabel('Количество сообщений')
plt.ylabel('Количество вхождений')

plt.show()
```



Для наглядности уберем лимит сообщений для ultra

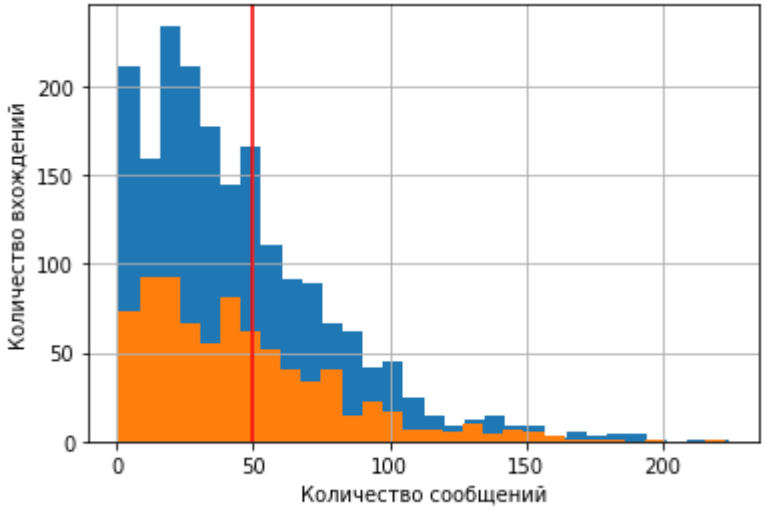
Ввод [76]:

```
messages_pivot[messages_pivot.tariff_name == 'smart']['count']['message_date'].hist(
messages_pivot[messages_pivot.tariff_name == 'ultra']['count']['message_date'].hist(

plt.axvline(x=int(tariffs.query('tariff_name == "smart"')['messages_included']), col
# plt.axvline(x=int(tariffs.query('tariff_name == "ultra"')['messages_included']), c

plt.xlabel('Количество сообщений')
plt.ylabel('Количество вхождений')

plt.show()
```



АНАЛИТИКА ДЛЯ SMART

Здесь видна сильная положительная скошенность

Три пика - на 10, 24 и 45 минутах

Меньше людей используют сообщения вне лимита, однако количество все еще внушительное

АНАЛИТИКА ДЛЯ ULTRA

Сильная отрицательная скошенность - люди на этом тарифе не любят отправлять сообщения

Большинство людей ограничевается 50 сообщениями в месяц, крайне малое количество - 100 сообщениями

До включенных в тариф 1000, опять же, никто не дошел

7.3. Интернет

Для smart

Ввод [77]:

```
print('Среднее значение:')
print(internet_pivot[internet_pivot.tariff_name == 'smart']['rounded_gb_used'].mean(
print('Дисперсия:')
print(internet_pivot[internet_pivot.tariff_name == 'smart']['rounded_gb_used'].var()
print('Стандартное отклонение:')
print(internet_pivot[internet_pivot.tariff_name == 'smart']['rounded_gb_used'].var()
print('Доступно бесплатно:', '\n15')
```

```
Среднее значение:
17.232495511669658

Дисперсия:
55.25261218004271

Стандартное отклонение:
7.433210085827167

Доступно бесплатно:
15
```

Для ultra

Ввод [78]:

```
print('Среднее значение:')
print(internet_pivot[internet_pivot.tariff_name == 'ultra']['rounded_gb_used'].mean(
print('Дисперсия:')
print(internet_pivot[internet_pivot.tariff_name == 'ultra']['rounded_gb_used'].var()
print('Стандартное отклонение:')
print(internet_pivot[internet_pivot.tariff_name == 'ultra']['rounded_gb_used'].var()
print('Доступно бесплатно:', '\n30')
```

```
Среднее значение:
17.646153846153847

Дисперсия:
50.82640972990049

Стандартное отклонение:
7.129264318981342

Доступно бесплатно:
30
```

Гистограмма

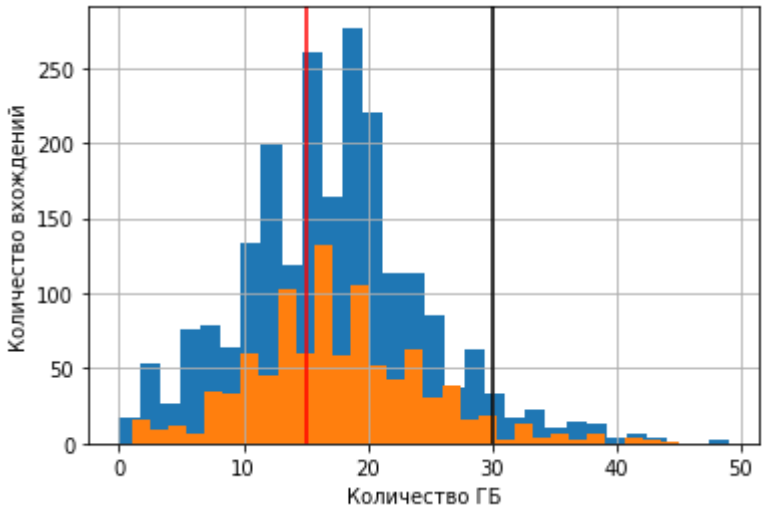
Ввод [79]:

```
internet_pivot[internet_pivot.tariff_name == 'smart']['rounded_gb_used'].hist(bins=3
internet_pivot[internet_pivot.tariff_name == 'ultra']['rounded_gb_used'].hist(bins=3

plt.axvline(x=int(tariffs.query('tariff_name == "smart"')['mb_per_month_included']//
plt.axvline(x=int(tariffs.query('tariff_name == "ultra"')['mb_per_month_included']//

plt.xlabel('Количество ГБ')
plt.ylabel('Количество вхождений')

plt.show()
```



АНАЛИТИКА ДЛЯ SMART

Распределение почти нормальное, с заметными пиками

Пики приходятся на крайние 14 Гб, но большинство докупает 5 Гб и дотягивает их

Малое количество людей используют интернет до 14 и после 20 Гб

АНАЛИТИКА ДЛЯ ULTRA

Здесь почти нормальное распределение, с большими пиками на различных значениях

Самый большой пик - на 20 Гб, после 30 Гб мало людей используют интернет

7.4. Общий вывод

На тарифе smart все услуги имеют превышения лимита и значительная часть людей покупает допольнительные пакеты

На тарифе ultra минуты и звонки по полной не использует никто, однако заметная часть людей покупает дополнительные Гб

8. Гипотезы

8.1. Ручные рассчеты выручки

8.1.1. Средняя выручка звонков сверх лимита

Посмотрим на звонки всех абонентов

Ввод [80]:

```
calls_pivot['full_fee'] = calls_pivot['rub_monthly_fee'] + calls_pivot['fee_over']
calls_pivot.head()
```

Out[80]:

		sum	count	tariff_name	minutes_included	rub_monthly_
		rounded_duration	rounded_duration			
month	user_id					
	1005	95	15	smart	500	
	1032	380	61	smart	500	
1	1045	9	1	smart	500	
	1055	551	64	smart	500	
	1071	194	24	smart	500	

Разделим их по тарифам

Ввод [81]:

```
calls_pivot_smart = calls_pivot[calls_pivot['tariff_name'] == 'smart']
calls_pivot_ultra = calls_pivot[calls_pivot['tariff_name'] == 'ultra']
```

Выведем сренюю помесячную для смарта

Ввод [82]:

```
display(calls_pivot_smart.pivot_table(index='month', values='fee_over', aggfunc='mea
```

```
month
1       4.500000
2      44.482759
3     103.480519
4      80.651786
5      82.531915
6      88.766871
7     103.358209
8     122.564103
9     129.544402
10    133.382253
11    131.961905
12    204.526786
Name: fee_over, dtype: float64
```

И так же для ультра

Ввод [83]:

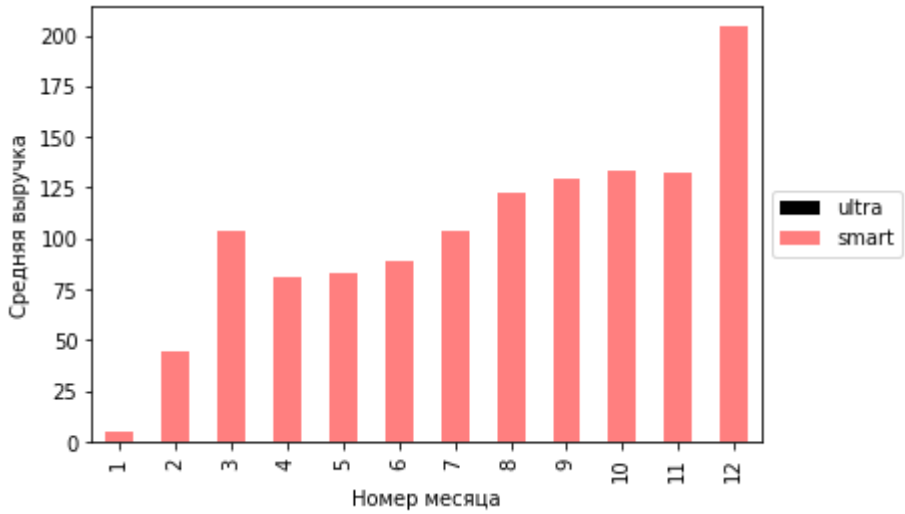
```
display(calls_pivot_ultra.pivot_table(index='month', values='fee_over', aggfunc='mea
```

```
month
1     0
2     0
3     0
4     0
5     0
6     0
7     0
8     0
9     0
10    0
11    0
12    0
Name: fee_over, dtype: int64
```

Построим барплот с ними обоими и сравним

Ввод [84]:

```
calls_pivot_ultra.pivot_table(index='month',
                              values='fee_over',
                              aggfunc='mean').plot(kind='bar',
                                                   color='black',
                                                   label='ultra',
                                                   width=1/3)
calls_pivot_smart.pivot_table(index='month',
                              values='fee_over',
                              aggfunc='mean').plot(kind='bar',
                                                   color='red',
                                                   label='smart',
                                                   alpha=0.5)
plt.xlabel('Номер месяца')
plt.ylabel('Средняя выручка')
plt.legend(bbox_to_anchor=(1, 0.6))
plt.show()
```



Как видно, пользователи тарифа ultra вообще не используют минуты сверх лимита, в то время как за год переплаты абонентов тарифа smart увеличились

8.1.2. Средняя выручка сообщений сверх лимита

Действуем по той же схеме

Ввод [85]:

```
messages_pivot['full_fee'] = messages_pivot['rub_monthly_fee'] + messages_pivot['fee
messages_pivot.head()
```

Out[85]:

		count	tariff_name	messages_included	rub_monthly_fee	rub_per_messa
		message_date				
month	user_id					
	1005	7	smart	50	550	
	1032	38	smart	50	550	
1	1045	5	smart	50	550	
	1055	30	smart	50	550	
	1071	16	smart	50	550	

Ввод [86]:

```
messages_pivot_smart = messages_pivot[messages_pivot['tariff_name'] == 'smart']
messages_pivot_ultra = messages_pivot[messages_pivot['tariff_name'] == 'ultra']
```

Ввод [87]:

```
display(messages_pivot_smart.pivot_table(index='month', values='fee_over', aggfunc='
```

```
month
1       6.157895
2       6.777778
3      23.268293
4      20.545455
5      25.218750
6      29.727273
7      35.338983
8      37.277512
9      34.663212
10     42.932432
11      0.000000
12      0.000000
Name: fee_over, dtype: float64
```

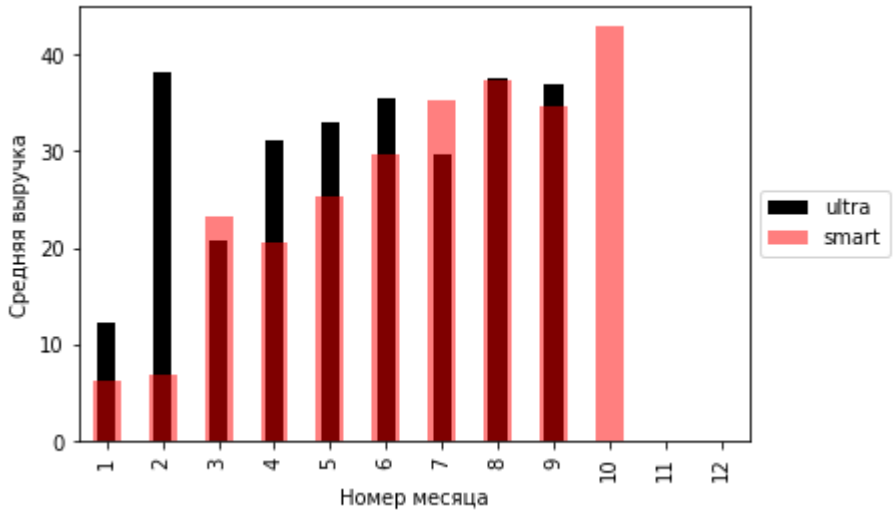
Ввод [88]:

```
display(messages_pivot_ultra.pivot_table(index='month', values='fee_over', aggfunc='
```

```
month
2     12.250000
3     38.250000
4     20.750000
5     31.043478
6     32.909091
7     35.400000
8     29.566265
9     37.608000
10    36.866667
11     0.000000
12     0.000000
Name: fee_over, dtype: float64
```

Ввод [89]:

```
messages_pivot_ultra.pivot_table(index='month',
                                 values='fee_over',
                                 aggfunc='mean').plot(kind='bar',
                                                      color='black',
                                                      label='ultra',
                                                      width=1/3)
messages_pivot_smart.pivot_table(index='month',
                                 values='fee_over',
                                 aggfunc='mean').plot(kind='bar',
                                                      color='red',
                                                      label='smart',
                                                      alpha=.5)
plt.xlabel('Номер месяца')
plt.ylabel('Средняя выручка')
plt.legend(bbox_to_anchor=(1, 0.6))
plt.show()
```



Здесь картина менее однозначная: в два последних месяца вообще никто не превышал лимиты, и в начале года тариф ultra превышал по выручке тариф smart

8.1.3. Средняя выручка интернета сверх лимита

Ввод [90]:

```
internet_pivot['full_fee'] = internet_pivot['rub_monthly_fee'] + internet_pivot['fee
internet_pivot.head()
```

Out[90]:

		sum mb_used	rounded_gb_used	tariff_name	gb_per_month_included	rub_monthly_fe
month	user_id					
	1005	2727.97	3	smart	15	55
	1032	24127.69	24	smart	15	55
1	**1045**	2346.71	3	smart	15	55
	1055	17090.88	17	smart	15	55
	1071	15604.34	16	smart	15	55

Ввод [91]:

```
internet_pivot_smart = internet_pivot[internet_pivot['tariff_name'] == 'smart']
internet_pivot_ultra = internet_pivot[internet_pivot['tariff_name'] == 'ultra']
```

Ввод [92]:

```
display(internet_pivot_smart.pivot_table(index='month', values='fee_over', aggfunc='
```

```
month
1     195.348837
2     321.428571
3     810.638298
4     552.755906
5     749.333333
6     867.096774
7     779.787234
8     841.176471
9     735.200000
10    803.846154
11     44.769231
12     69.421488
Name: fee_over, dtype: float64
```

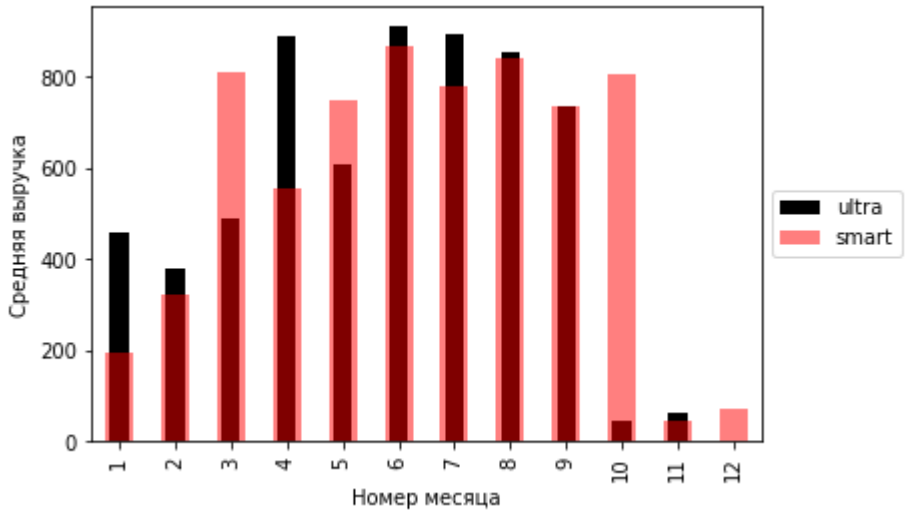
Ввод [93]:

```
display(internet_pivot_ultra.pivot_table(index='month', values='fee_over', aggfunc='
```

```
month
2     458.333333
3     380.000000
4     490.000000
5     887.719298
6     606.521739
7     910.679612
8     891.176471
9     851.612903
10    734.265734
11     45.967742
12     62.946429
Name: fee_over, dtype: float64
```

Ввод [94]:

```
internet_pivot_ultra.pivot_table(index='month',
                                 values='fee_over',
                                 aggfunc='mean').plot(kind='bar',
                                                      color='black',
                                                      label='ultra',
                                                      width=1/3)
internet_pivot_smart.pivot_table(index='month',
                                 values='fee_over',
                                 aggfunc='mean').plot(kind='bar',
                                                      color='red',
                                                      label='smart',
                                                      alpha=.5)
plt.xlabel('Номер месяца')
plt.ylabel('Средняя выручка')
plt.legend(bbox_to_anchor=(1, 0.6))
plt.show()
```



8.1.4. Вывод

Выручка действительно отличается, но на тарифе smart она растет, а на ultra лишь интернет создает разницу в выручках помесячно

8.2. Вероятностные рассчеты выручки

8.2.1. Гипотеза

```
Обозначим нулевую гипотезу:

- Средняя выручка пользователей тарифов "Ультра" и "Смарт" различаются

Тогда альтернативная будет:

- Средняя выручка пользователей тарифов "Ультра" и "Смарт" не различаются

Если не подтвердится нулевая гипотеза, соответственно, выручка с
пользователей разных тарифов не будет различаться, то за истинную мы примем
альтернативную гипотезу
```

8.2.2. Выручка с каждого пользователя за месяц

Ввод [95]:

```
# добавляем тариф каждого пользователя
all_users_pivot['tariff'] = (all_users_pivot
 .merge(users[['user_id', 'tariff']], on='user_id', how='left')
 .merge(tariffs[['tariff_name', 'rub_monthly_fee']], left_on='tariff', right_on='tar
 .tolist())

# добавляем ежемесячную плату
all_users_pivot['monthly_fee'] = (all_users_pivot
 .merge(tariffs[['tariff_name', 'rub_monthly_fee']], left_on='tariff', right_on='tar
 .tolist())

# добавляем город пользователя
all_users_pivot['city'] = (all_users_pivot
 .merge(users[['user_id', 'city']], on='user_id', how='left')['city']
 .tolist())

# рассчитываем доход от пользователя за месяц
all_users_pivot['month_fee'] = all_users_pivot['monthly_fee'] + all_users_pivot['fee

all_users_pivot
```

Out[95]:

		fee_over_calls	fee_over_calls	fee_over	fee_over_sum	tariff	monthly_fee	
user_id	**month**							
	5	0.0	0.0	0.0	0.0	ultra	1950	Крас
	6	0.0	30.0	1600.0	1630.0	ultra	1950	Крас
1000	**7**	0.0	75.0	0.0	75.0	ultra	1950	Крас
	8	0.0	93.0	0.0	93.0	ultra	1950	Крас
	9	0.0	21.0	0.0	21.0	ultra	1950	Крас
...	**...**	...	...	...	...	...	...	
	10	0.0	0.0	0.0	0.0	smart	550	
1499	**11**	336.0	0.0	0.0	336.0	smart	550	
	12	0.0	0.0	0.0	0.0	smart	550	
1128	**1**	0.0	0.0	0.0	0.0	smart	550	Вол
1371	**1**	0.0	0.0	0.0	0.0	smart	550	

3216 rows × 8 columns

Ввод [96]:

```
smart_users_pivot = all_users_pivot.query('tariff == "smart"')
ultra_users_pivot = all_users_pivot.query('tariff == "ultra"')

smart_users_month_fee = smart_users_pivot['month_fee'].tolist()
ultra_users_month_fee = ultra_users_pivot['month_fee'].tolist()
```

8.2.3. Расчеты

Ввод [97]:

```
results = st.ttest_ind(smart_users_month_fee,
                       ultra_users_month_fee,
                       equal_var = False)
```

Ввод [98]:

```
results.pvalue
```

Out[98]:

```
3.8540514286984096e-288
```

8.2.4. Вывод

```
Значения очень малы, так что при любом alpha гипотеза оказыается
подтверждена
```

8.3. Вероятностные расчеты регионов

8.3.1. Гипотеза

```
Нулевая гипотеза звучит следующим образом:

- Средняя выручка пользователей из Москвы отличается от выручки
пользователей из других регионов

Альтернативную гипотезу сформулируем так:

- Средняя выручка пользователей из Москвы не отличается от выручки
пользователей из других регионов

Если не подтвердится нулевая гипотеза, соответственно, выручка с
пользователей из Москвы не будет отличаться от выручки с пользователей из
регионов, то за истинную мы примем альтернативную гипотезу
```

8.3.2. Расчеты

Ввод [99]:

```
moscow_users_pivot = all_users_pivot.query('city == "Москва"')
regions_users_pivot = all_users_pivot.query('city != "Москва"')

moscow_users_pivot_fee = moscow_users_pivot['month_fee'].tolist()
regions_users_pivot_fee = regions_users_pivot['month_fee'].tolist()
```

Ввод [100]:

```
regions_results = st.ttest_ind(moscow_users_pivot_fee,
                        regions_users_pivot_fee,
                        equal_var = False)
```

Ввод [101]:

```
regions_results.pvalue
```

Out[101]:

```
0.40844519197438356
```

8.3.3. Вывод

```
Очень высокий показатель p-value, вследствие чего мы принимаем за истину
нулевую гипотезу
```

9. Общий вывод

9.1. Полученные данные

```
Данные оказались чистыми, без ошибок и аномальных значений

Даты приведены к нужному формату, лишние столбцы выкинуты
```

9.2. Анализ полученных данных

```
Тариф smart разумно подходит к своим лимитам - большинство людей остаются в
их границах, используя по полной

Однако, и процет людей, приносящих дополнительную прибыль посредством
покупки дополнительных пакетов, высок

Тариф ultra имеет излишний лимит сообщений и минут, который не расходуют
даже на половину

На тарифе ultra разумным виден лимит интернета, так как достаточное
количество людей докупают дополнительные Гб
```

9.3. Гипотезы

9.3.1. Выручка по тарифам различается

9.3.1. Выручка на тарифах различается

Это действительно так, выручка на тарифе smart и на тарифе ultra различается – как правило, со smart выручка больше, что доказывают и ручные рассчеты, и встроенные решения

9.3.2. Выручка по регионам отличается от Москвы

Эта гипотеза не подтвердилась, выручка по регионам отличается не так сильно

9.4. Итог

Наилучшим тарифом является тариф smart, так как там более точно подобраны